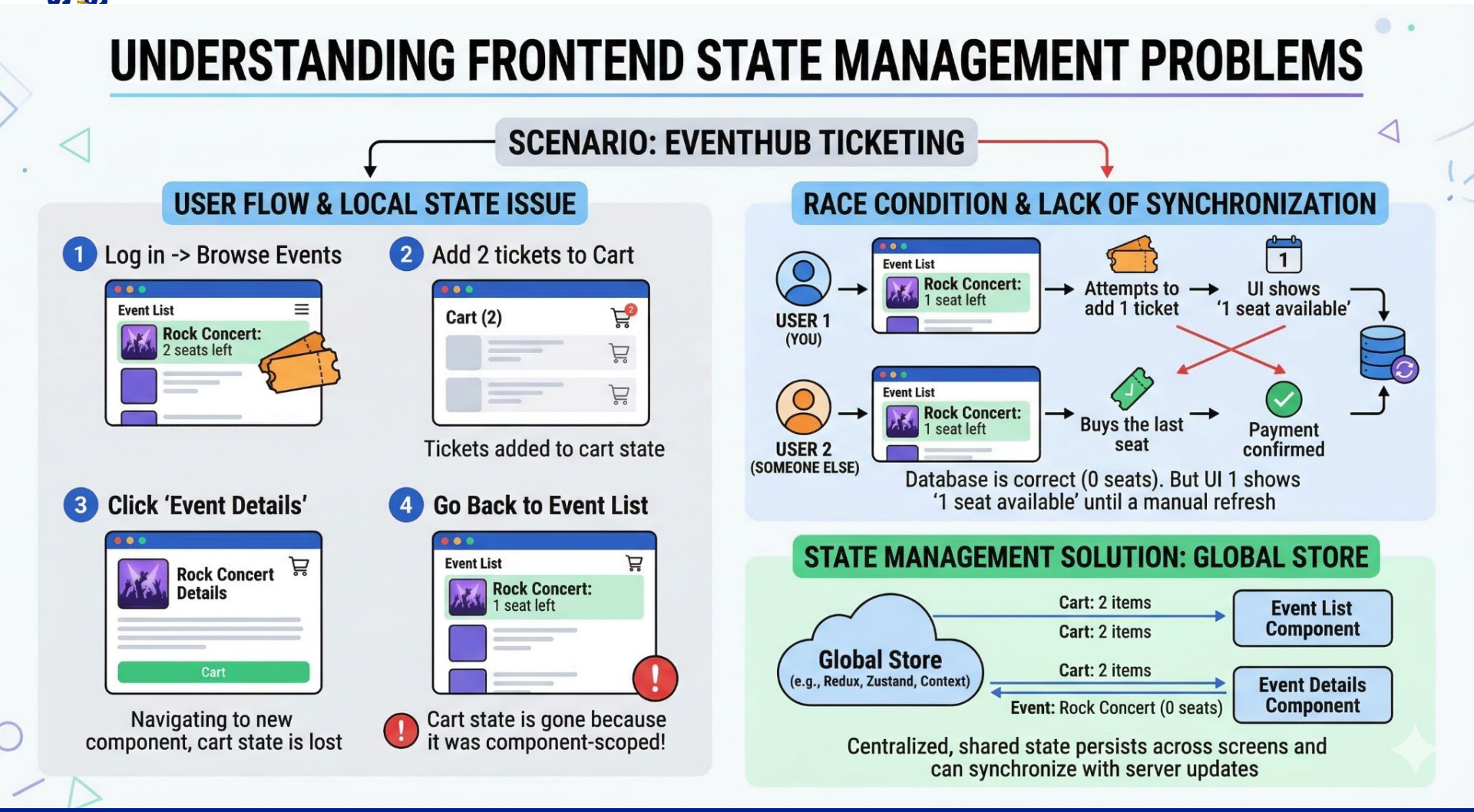
UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

Software Architectures

Practice 6

Zoltán Richárd JÁNKI, PhD

Scenario: EventHub



What is State?

- State = any data that can change over time and affects what the user sees.
- Examples from EventHub:
  - The list of upcoming events
  - Whether the user is logged in
  - The contents of the shopping cart
  - Which filter is currently active
  - Whether a modal dialog is open or closed

Categories of State

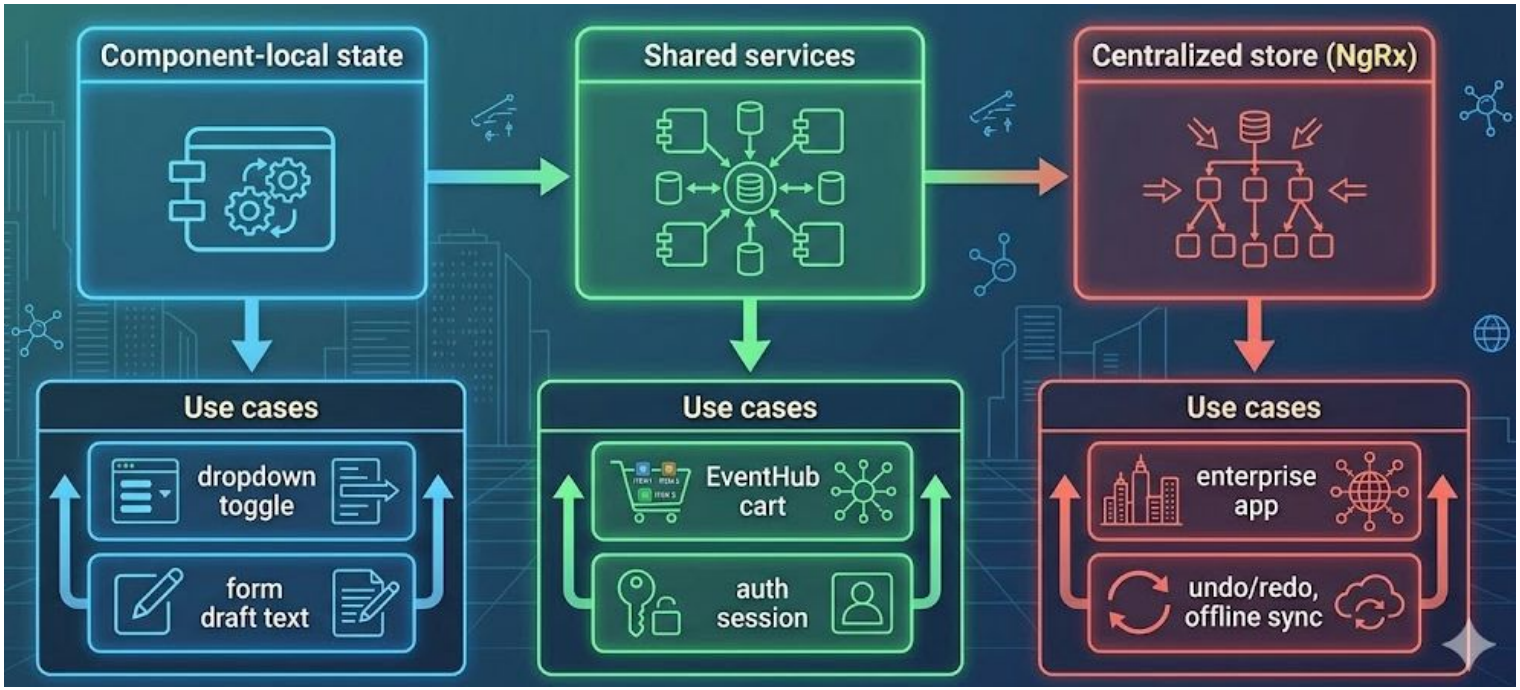
Category	Source of truth	EventHub example
Server state	Database	Event list, booking history, user profile
Client state	Browser only	Sidebar open/closed, active tab, search input text
Synchronized state	Originates on server, must stay fresh on client	Cart contents, “seats remaining” counter, auth session

- Server state → ✅
- Client state → ✅
- Synchronized state → ❌

Single Source of Truth principle

- For any piece of state, there should be exactly ONE authoritative place where it is stored and updated.
- What goes wrong without it:
  - Component A and Component B each hold their own copy of the cart
  - User adds an item → Component A updates, Component B doesn't
  - The UI becomes inconsistent

Complexity spectrum



- Pick the simplest approach that solves your problem!
  - over-engineering ❌
  - under-engineering ❌

Approach 1: Component-local state

- Each component manages its own data.
  - If the state is only relevant to one component — a form input, a toggle, an animation flag — keep it here.
  - The moment a second component needs the same data, you've outgrown this approach.

- Example (EventHub)
  - cart data
    - EventListComponent (owner)
    - HeaderComponent (must know about it)
- Passing data
  - e.g.: @Input() and @Output() in Angular

UNIVERSITAS SCIENTIARUM SZEGEDIENSIS
UNIVERSITY OF SZEGED
Department of Software Engineering

Approach 2: Shared Services

- Service = a so-called singleton class, injected wherever needed
  - exactly one instance is present for the entire application
  - every component that injects it, gets the same object
  - a single source of truth

- Example (EventHub)
  - cart data
    - EventListComponent calls cartService.addToCart()
    - polling the service → ❌
    - manually call a refresh() method of HeaderComponent →❌
    - the service notifies all interested components →reactive ✅
  - Passing data
    - through subscriptions

UNIVERSITY OF SZEGED
Department of Software Engineering
UNIVERSITAS SCIENTIARUM SZEGEDIENSIS

Approach 3: Centralized Store

- Store = a single place where the entire application state lives
  - everyone reads from this board (nobody modifies it directly)
- Action = describes what happened, nothing more
  - nothing about how to handle it
- Reducer = a pure function that takes the current state of the store and produces a new version of it including the changes
- Selector = extracts a “slice” of the state
- Effect = listens for specific actions and creates a new action

- Example (EventHub) - NgRx
  - cart data
    - component creates an action: { type: 'Add To Cart', payload: { eventId: 42, title: 'Concert' } }
    - reducer receives the action, takes the current state and return a new updated state (old state is untouched)
    - components that use selectors, automatically receives the update

Approach +1: Signals

- Signal = an object that holds a value that automatically notifies anything that reads it when the value changes

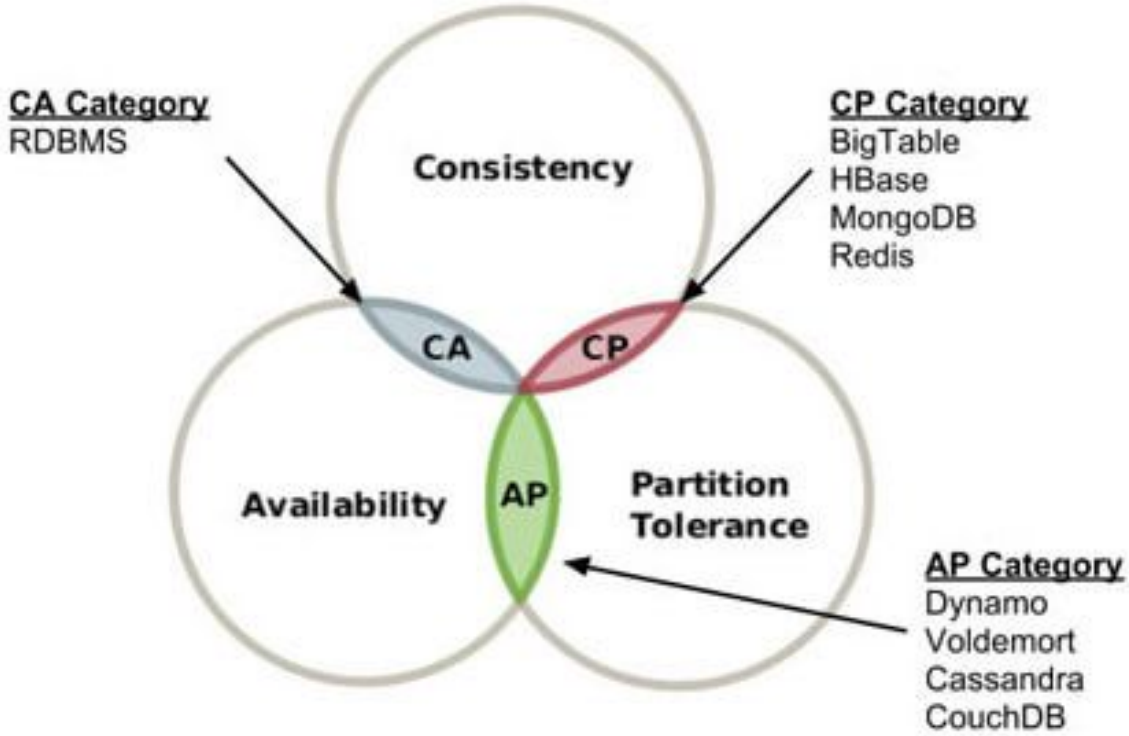
- HTTP requests are still used with Observables
- Signals make local state-management easier

```
@Injectable({ providedIn: 'root' })
export class CartService {
  private cartItems = signal<CartItem[]>([]);

  readonly cart = this.cartItems.asReadonly();
  readonly total = computed(() =>
    this.cartItems().reduce((sum, i) => sum + i.price * i.quantity, 0)
  );

  addToCart(item: CartItem) {
    this.cartItems.update(items => [...items, item]);
  }
}
```

Choosing the right approach

- Which is best? ❌
- Which is appropriate for my project? ✅

Situation	Approach	Example
UI-only, one component	Component property	Is dropdown open?
Shared across a few components	Service + BehaviorSubject *or* Signals	Cart, auth state
Complex interactions, large app	NgRx (centralized store)	Enterprise dashboard

Distributed systems

CAP Theorem